

Career Guide Dashboard - Final Project Report

1. Project Title

Career Guide Dashboard

The report uses Harvard-style in-text citations to support the technical choices, design decisions, deployment approach, and security practices used in the project.

2. Project Overview

The Career Guide Dashboard is a full-stack web application designed to help students and job seekers explore career domains, understand job market demand, compare salary ranges, save opportunities, and receive career recommendations. The system combines a modern React frontend, a FastAPI backend, MongoDB database storage, authentication, external job data integration, and resume-based recommendation features (Meta Open Source, 2026; FastAPI, 2026; MongoDB, 2026a).

The project was built to solve a common problem faced by students: choosing a suitable career path without having a single place to compare job availability, salary expectations, future scope, personal skills, saved jobs, and application progress. This dashboard brings those activities into one organized platform.

3. Purpose of the Dashboard

The main purpose of the Career Guide Dashboard is to support better career decision-making. Instead of only showing static career information, the dashboard uses real job data, salary information, user activity, and resume analysis to guide users toward suitable career paths (Adzuna, 2026; Mozilla, 2026a).

The dashboard helps users:

- Search career domains such as Data Science, Cloud, Artificial Intelligence, Cyber Security, and related roles.
- View job demand and salary information for selected career domains.
- Receive a career score and future scope prediction based on job availability and salary range.
- Save favorite careers and jobs for later review.
- Upload a resume and receive skill-based career recommendations.
- Track applied jobs and profile activity.
- Access the platform through a deployed web application.

In simple terms, the dashboard acts as a career planning assistant. It gives users a clearer view of which career options are currently strong in the job market and which skills they may need to improve.

4. Problem Statement

Students and early-career professionals often struggle to choose a career direction because information is spread across many platforms. Job listings, salaries, skill requirements, and career predictions are usually not available in one place. This can make career planning confusing and time-consuming.

The Career Guide Dashboard addresses this issue by providing a centralized platform where users can search careers, analyze job trends, view salary ranges, save jobs, and compare personal resume skills with career requirements (Adzuna, 2026; MongoDB, 2026a).

5. Objectives

The major objectives of this project are:

- To build an interactive dashboard for career exploration (React Router, 2026; MUI, 2026).
- To integrate real job data using an external job API (Adzuna, 2026).
- To calculate career predictions using job count and salary data (Mozilla, 2026a).
- To provide secure user registration and login (Jones, Bradley and Sakimura, 2015; OWASP Foundation, 2026a).
- To allow users to save favorite careers and jobs.
- To support resume upload and skill-based recommendations.
- To deploy the project online using Render and Vercel (Render, 2026; Vercel, 2026a).
- To create a responsive user interface that works across desktop and mobile screens (W3C, 2023; Mozilla, 2026b).

6. Technology Stack

Frontend

- **React:** Used to build the user interface (Meta Open Source, 2026).
- **Vite:** Used for fast frontend development and production build (Vite, 2026).
- **Material UI:** Used for dashboard cards, icons, buttons, charts, and layout elements (MUI, 2026).
- **React Router:** Used for page navigation (React Router, 2026).
- **Axios:** Used to communicate with the backend API (Axios, 2026).
- **CSS:** Used for custom styling and responsive layouts (Mozilla, 2026b; W3C, 2023).

Backend

- **FastAPI:** Used to build REST API endpoints (FastAPI, 2026).
- **Uvicorn:** Used as the ASGI server for running the backend (Encode OSS, 2026).
- **Python:** Main backend programming language (Python Software Foundation, 2026).
- **Motor / PyMongo:** Used to connect FastAPI with MongoDB asynchronously (MongoDB, 2026b; MongoDB, 2026c).
- **JWT Authentication:** Used to protect user-specific routes (Jones, Bradley and Sakimura, 2015).
- **Passlib / bcrypt:** Used for password hashing (Passlib, 2020; Provos and Mazieres, 1999).

Database

- **MongoDB Atlas:** Used as the cloud database for storing users, jobs, favorites, resume profiles, and application data (MongoDB, 2026a).

External API

- **Adzuna Job API:** Used to fetch live job listings, salary information, company details, job descriptions, and locations (Adzuna, 2026).

Deployment

- **Render:** Used to deploy the FastAPI backend (Render, 2026).
- **Vercel:** Used to deploy the React frontend (Vercel, 2026a).
- **GitHub:** Used for version control and connecting the codebase to deployment platforms (GitHub, 2026).

7. System Architecture

The application follows a client-server architecture, with the frontend and backend separated through HTTP API calls in line with common web application and REST design principles (Fielding, 2000; Mozilla, 2026a).

1. The user interacts with the React frontend deployed on Vercel.
2. The frontend sends API requests to the FastAPI backend deployed on Render.
3. The backend validates requests, handles authentication, processes data, and communicates with MongoDB Atlas (MongoDB, 2026a; Pydantic, 2026).
4. For career and job search features, the backend also fetches job data from the Adzuna API (Adzuna, 2026).
5. The backend returns structured JSON responses to the frontend (ECMA International, 2017).
6. The frontend displays the results as dashboard cards, job lists, charts, recommendations, and profile information.

8. Main Modules and Features

8.1 Authentication Module

The authentication module allows users to register and log in securely. Passwords are hashed before being stored in the database, which follows password storage guidance that recommends one-way hashing rather than plaintext storage (OWASP Foundation, 2026a). After login, the system generates a JWT access token. This token is used to access protected features such as dashboard data, saved jobs, profile details, resume analysis, and job application tracking (Jones, Bradley and Sakimura, 2015).

Key features:

- User registration
- User login
- Password hashing

- JWT token generation
- Protected backend routes

8.2 Dashboard Module

The dashboard module gives users a summary of their activity and career exploration progress. It displays the user profile name, role, saved careers, saved jobs, and total jobs available in the database. The dashboard also includes visual cards and charts to make the data easier to understand, supporting common usability principles of visibility and clear feedback (Nielsen Norman Group, 2024).

Key features:

- Total job count
- Saved career count
- Saved job count
- User role display
- Visual progress and activity summary
- Resume recommendation section

8.3 Career Search Module

The career search module allows users to search career domains such as Data Science, Cloud, Cyber Security, and Artificial Intelligence. The backend checks existing database records and fetches new job data from the Adzuna API when needed. It then calculates useful career insights (Adzuna, 2026; MongoDB, 2026a).

Key features:

- Career domain search
- Related role expansion
- Job data import from Adzuna
- Total jobs calculation
- Average salary calculation
- Career score and recommendation generation

8.4 Career Prediction Module

The prediction module calculates the future scope of a career based on job demand and average salary. The algorithm assigns points for job count, salary range, and demand level. Based on the final score, the career is classified into categories such as Excellent, Very High, High, Moderate, or Low. This rule-based approach keeps the recommendation logic explainable for students and early-career users (Molnar, 2022).

Prediction factors:

- Number of available jobs

- Average salary
- Demand score

Example output:

- Career score
- Future scope
- Recommendation message

8.5 Jobs Module

The jobs module displays available job opportunities. Users can view job title, company, domain, location, contract type, salary range, and description. The system also allows authenticated users to add manual jobs and mark jobs as applied (Adzuna, 2026; FastAPI, 2026).

Key features:

- View saved and imported jobs
- Import jobs from external API
- Add manual jobs
- Mark jobs as applied
- Track application count
- View job requirements and salary details

8.6 Favorites Module

The favorites module allows users to save important career domains and job opportunities. This gives users a personalized list of careers and jobs they want to revisit later, which supports user control and task continuity in interactive systems (Nielsen Norman Group, 2024).

Key features:

- Save favorite careers
- Save favorite jobs
- View favorite careers
- View favorite jobs
- Delete saved favorites

8.7 Resume Analysis Module

The resume analysis module allows users to upload a resume in PDF, DOCX, or TXT format. The backend extracts resume text, detects skills, compares them with predefined career profiles, and generates recommendations. The use of common document formats supports accessibility and portability for users who store resumes in different file types (Adobe, 2026; Microsoft, 2026).

The module identifies skills related to areas such as:

- Software development
- Data analysis
- Data science
- Cloud engineering
- Cybersecurity
- UI/UX design
- Project management

Key features:

- Resume upload
- PDF, DOCX, and TXT support
- Skill extraction
- Career matching
- Match score calculation
- Skills-to-learn suggestions
- Related job recommendations

8.8 Profile Module

The profile module displays user details, resume analysis results, matched skills, and applied jobs. This helps users track their career preparation progress from one page, keeping important user-specific information in a single protected view (OWASP Foundation, 2026b).

Key features:

- View user name, email, and role
- View latest resume profile
- View detected resume skills
- View applied jobs
- Count applied jobs

9. Database Design

The project uses MongoDB collections to store application data (MongoDB, 2026a).

Main collections:

- **users:** Stores user information, hashed passwords, roles, and resume profile data.
- **jobs:** Stores imported and manually added jobs.

- **favorites:** Stores saved careers and saved jobs for each user.
- **career_domains:** Used for career-related domain data.

MongoDB was selected because the project handles flexible JSON-like data such as job descriptions, user profiles, resume results, and API responses (MongoDB, 2026a; ECMA International, 2017).

10. API Design

The backend exposes REST API endpoints using FastAPI, with JSON request and response bodies and a resource-based route structure (FastAPI, 2026; Fielding, 2000).

Important API groups:

- `/auth`: Registration and login
- `/dashboard`: User dashboard summary
- `/search`: Career search and job market analysis
- `/predictions`: Career future scope prediction
- `/jobs`: Job listing, manual job creation, and apply tracking
- `/favorites`: Save, view, and delete favorites
- `/profile`: User profile and applied jobs
- `/resume`: Resume upload and analysis

The API uses JSON responses and JWT-based route protection for user-specific operations (ECMA International, 2017; Jones, Bradley and Sakimura, 2015).

11. User Interface Design

The frontend is designed as a responsive dashboard application. It includes a sidebar navigation system and separate pages for Home, Dashboard, Career Search, Jobs, Favorites, Profile, Login, and Register (React Router, 2026; W3C, 2023).

The interface focuses on:

- Clear navigation
- Dashboard cards
- Visual career insights
- Search and recommendation flow
- Responsive layouts for different screen sizes
- Simple forms for login, registration, job creation, and resume upload

The user interface was designed to make career data easier to understand through summaries, charts, cards, and organized sections. Accessibility and responsive design principles were considered so that the interface remains usable across different screen sizes and user contexts (W3C, 2023; Mozilla, 2026b).

12. Deployment

The project was deployed using Render and Vercel, separating backend service hosting from frontend static application hosting (Render, 2026; Vercel, 2026a).

Backend Deployment

The FastAPI backend is deployed on Render (Render, 2026).

Backend URL:

`https://career-guide-backend-a1ib.onrender.com`

Render runs the backend using:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Frontend Deployment

The React frontend is deployed on Vercel (Vercel, 2026a).

Frontend URL:

`https://web-development-project-omega.vercel.app/`

The frontend communicates with the backend using the environment variable, following configuration practices that separate deploy-specific settings from source code (Wiggins, 2017):

```
VITE_API_BASE_URL=https://career-guide-backend-a1ib.onrender.com
```

CORS Configuration

The backend uses configurable CORS settings so that the deployed Vercel frontend can securely communicate with the Render backend (Mozilla, 2026c).

13. Testing and Validation

The project was tested through:

- Frontend production build using Vite (Vite, 2026).
- Backend syntax validation (Python Software Foundation, 2026).
- Render backend deployment logs (Render, 2026).
- API root endpoint verification.
- Vercel frontend deployment (Vercel, 2026a).
- Manual testing of user workflows such as registration, login, search, jobs, and dashboard access.

The backend deployment was successful, and Render confirmed that the service was live. The frontend was also deployed successfully through Vercel.

14. Security Considerations

Security was considered in the following ways:

- Passwords are hashed before storage (OWASP Foundation, 2026a).

- JWT tokens are used for authenticated access (Jones, Bradley and Sakimura, 2015).
- Environment variables are used for sensitive values (Wiggins, 2017).
- Real secrets are not intended to be committed to GitHub.
- CORS is configured to allow only approved frontend origins (Mozilla, 2026c).
- User-specific data is filtered using the logged-in user's email.

For production use, API keys, database passwords, and JWT secrets should be rotated if they were exposed during development, and the application should continue to follow secure design practices for authentication and access control (OWASP Foundation, 2026b).

15. Challenges Faced

Some challenges in the project included:

- Connecting the frontend and backend across separate deployment platforms.
- Managing environment variables for local and production environments.
- Handling CORS between Vercel and Render.
- Extracting readable text from different resume file formats.
- Converting external job API data into a consistent database structure.
- Designing prediction logic that is understandable and useful for students.
- Making the dashboard responsive and user-friendly.

16. Future Enhancements

The project can be improved further by adding:

- More advanced machine learning for career prediction.
- Admin dashboard for managing jobs and career domains.
- Email notifications for saved jobs.
- More detailed resume scoring.
- Job filtering by country, salary, remote work, and experience level.
- Charts using a dedicated chart library.
- Interview preparation resources based on selected career domain.
- Exportable PDF career reports for users.
- OAuth login using Google or GitHub.

17. Conclusion

The Career Guide Dashboard successfully provides a centralized platform for career exploration and planning. It combines job search, salary analysis, career prediction, favorites, resume analysis, and user profile tracking into one full-stack application.

The project demonstrates practical knowledge of frontend development, backend API design, database integration, authentication, external API usage, deployment, and responsive dashboard design (Meta Open Source, 2026; FastAPI, 2026; MongoDB, 2026a; Vercel, 2026a). It is useful for students and job seekers because it helps them make more informed career decisions based on real job data and personal skills.

Overall, this project meets its goal of creating a modern, interactive, and useful career guidance system.

18. References

- Adobe (2026) *PDF file format*. Available at: <https://www.adobe.com/acrobat/about-adobe-pdf.html> (Accessed: 6 July 2026).
- Adzuna (2026) *Adzuna API documentation*. Available at: <https://developer.adzuna.com/docs> (Accessed: 6 July 2026).
- Axios (2026) *Axios documentation*. Available at: <https://axios-http.com/docs/intro> (Accessed: 6 July 2026).
- ECMA International (2017) *ECMA-404: The JSON data interchange syntax*. Available at: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/> (Accessed: 6 July 2026).
- Encode OSS (2026) *Uvicorn documentation*. Available at: <https://www.uvicorn.org/> (Accessed: 6 July 2026).
- FastAPI (2026) *FastAPI documentation*. Available at: <https://fastapi.tiangolo.com/> (Accessed: 6 July 2026).
- Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. Doctoral dissertation. University of California, Irvine. Available at: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (Accessed: 6 July 2026).
- GitHub (2026) *GitHub Docs*. Available at: <https://docs.github.com/> (Accessed: 6 July 2026).
- Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Available at: <https://www.rfc-editor.org/rfc/rfc7519> (Accessed: 6 July 2026).
- Meta Open Source (2026) *React documentation*. Available at: <https://react.dev/> (Accessed: 6 July 2026).
- Microsoft (2026) *Open XML formats and file name extensions*. Available at: <https://support.microsoft.com/office/open-xml-formats-and-file-name-extensions-5200d93c-3449-4380-8e11-31ef14555b18> (Accessed: 6 July 2026).
- Molnar, C. (2022) *Interpretable Machine Learning: A guide for making black box models explainable*. 2nd edn. Available at: <https://christophm.github.io/interpretable-ml-book/> (Accessed: 6 July 2026).
- MongoDB (2026a) *MongoDB Atlas documentation*. Available at: <https://www.mongodb.com/docs/atlas/> (Accessed: 6 July 2026).
- MongoDB (2026b) *Motor documentation*. Available at: <https://motor.readthedocs.io/> (Accessed: 6 July 2026).
- MongoDB (2026c) *PyMongo documentation*. Available at: <https://pymongo.readthedocs.io/> (Accessed: 6 July 2026).
- Mozilla (2026a) *HTTP overview*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview> (Accessed: 6 July 2026).
- Mozilla (2026b) *Responsive design*. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design (Accessed: 6 July 2026).
- Mozilla (2026c) *Cross-Origin Resource Sharing (CORS)*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS> (Accessed: 6 July 2026).
- MUI (2026) *Material UI documentation*. Available at: <https://mui.com/material-ui/getting-started/> (Accessed: 6 July 2026).
- Nielsen Norman Group (2024) *10 usability heuristics for user interface design*. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/> (Accessed: 6 July 2026).
- OWASP Foundation (2026a) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 6 July 2026).
- OWASP Foundation (2026b) *Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Accessed: 6 July 2026).
- Passlib (2020) *Passlib documentation*. Available at: <https://passlib.readthedocs.io/> (Accessed: 6 July 2026).

- Provos, N. and Mazieres, D. (1999) *A future-adaptable password scheme*. Available at: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf> (Accessed: 6 July 2026).
- Pydantic (2026) *Pydantic documentation*. Available at: <https://docs.pydantic.dev/> (Accessed: 6 July 2026).
- Python Software Foundation (2026) *Python documentation*. Available at: <https://docs.python.org/3/> (Accessed: 6 July 2026).
- React Router (2026) *React Router documentation*. Available at: <https://reactrouter.com/> (Accessed: 6 July 2026).
- Render (2026) *Render documentation*. Available at: <https://render.com/docs> (Accessed: 6 July 2026).
- Vercel (2026a) *Vercel documentation*. Available at: <https://vercel.com/docs> (Accessed: 6 July 2026).
- Vite (2026) *Vite guide*. Available at: <https://vite.dev/guide/> (Accessed: 6 July 2026).
- W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. Available at: <https://www.w3.org/TR/WCAG22/> (Accessed: 6 July 2026).
- Wiggins, A. (2017) *The Twelve-Factor App: Config*. Available at: <https://12factor.net/config> (Accessed: 6 July 2026).